

Oberfläche

Organisation: Sie bearbeiten dieses Blatt in Ihrem Zweierteam im Repository aus Blatt 1. Wo eine Aufgabe Rollen nennt, übernimmt eine Person Rolle A und die andere Rolle B; in der siebten Aufgabe tauschen Sie. Voraussetzung sind beide Einheiten von T10. Sie arbeiten mit `freiraum-start-5.zip` aus Moodle oder mit Ihrem eigenen Stand nach Blatt 4. Aufgaben und Testat 5 verwenden die Klassennamen des Startstands. Alle Räume, Gruppen und Zeiten in den Beispieldaten sind frei erfunden.

Sie schreiben den Java-Code dieses Blatts ohne KI-Werkzeug und bedienen Git selbst, Befehl für Befehl. Im Testat und in der Vorführung erklären Sie jede Zeile und jeden Befehl.

Abgabe: Setzen Sie am Ende den Tag `abgabe-5`, packen Sie den Projektordner **mit** dem Ordner `.git` als ZIP und laden Sie ihn bis [OFFEN: Datum] über Moodle/APA hoch. Fehlermeldungen und Statusmeldungen kopieren Sie wörtlich aus Ihrem eigenen Lauf in `NOTIZEN.md`, Abschnitt „Blatt 5 · Schichtzuordnung“. Zur Vorführung starten Sie die Oberfläche mit `mvn javafx:run`.

Erste Aufgabe (40 Minuten): Spielen Sie den Startstand so ein, wie es `README.md` im ZIP beschreibt: Branch `start-5` anlegen, Dateien kopieren (auch `pom.xml`), `git diff --staged` lesen, committen, nach `main` mergen, pushen. Klicken Sie in IntelliJ danach im Maven-Fenster auf *Reload All Maven Projects*. Starten Sie die Oberfläche über das Maven-Fenster (*Plugins* › *javafx* › *javafx:run*) oder im Projektordner mit `mvn javafx:run`. Beim ersten Start lädt Maven OpenJFX 21.0.5 herunter. Es erscheint das Fenster „Freiraum“ aus diesem Gerüst:

```
@Override
public void start(Stage stage) {
    Label status = new Label("Freiraum läuft.");
    Button knopf = new Button("Buchen");
    VBox wurzel = new VBox(8, status, knopf);
    stage.setTitle("Freiraum");
    stage.setScene(new Scene(wurzel, 300, 120));
    stage.show();
}
```

Klicken Sie auf „Buchen“ und schreiben Sie in einem Satz in `NOTIZEN.md`, warum nichts passiert. Starten Sie `FreiraumApp` testweise über den grünen Pfeil neben `main`. Meldet Java Error: `JavaFX runtime components are missing, and are required to run this application`, tragen Sie die Meldung ein und starten die Oberfläche ab jetzt nur noch über Maven.

Legen Sie den Branch `umbau-pakete` an. Verschieben Sie alle Klassen aus dem Paket `freiraum` nach `freiraum.logik` (Klassen markieren, *Refactor* › *Move*) und `Buchungsterminal` nach `freiraum.oberflaeche`, denn ein Konsolenmenü ist Oberfläche. Für `freiraum.logik` gilt ab jetzt eine Regel: Keine Datei in diesem Paket importiert etwas aus `javafx`. Suchen Sie in `src/main/java/freiraum/logik` nach `import javafx` (IntelliJ: *Edit* › *Find* › *Find in Files*; Kommandozeile: `grep -rn "import javafx" src/main/java/freiraum/logik`). Die Suche findet nichts. Kompilieren Sie mit `mvn compile` und starten Sie ein Prüfprogramm ohne JavaFX:

```
java -cp target/classes freiraum.logik.PruefungDatei
```

Die letzte Zeile lautet `20 OK, 0 FEHLER`. Committen Sie, mergen Sie `umbau-pakete` nach `main` und pushen Sie.

Zweite Aufgabe (60 Minuten): Skizzieren Sie die Buchungsoberfläche zuerst auf Papier, und zwar zweimal: so, wie der Mensch sie sieht, und als Szenengraf, also als Baum mit der Wurzel oben. Die Oberfläche enthält:

- links eine `ListView<Raum>` mit den Räumen und rechts eine `ListView<String>` mit der Belegung des gewählten Raums, jede mit einem `Label` als Überschrift,
- eine Eingabezeile aus fünf `TextField` (Gruppe, Personen, Tag, Beginn, Ende) und einer `CheckBox` „Einweisung“,
- die Buttons „Buchen“ und „Belegung anzeigen“,
- darunter ein `Label` als Statuszeile.

Zeichnen Sie jeden Layout-Knoten (`VBox`, `HBox`) in den Baum, auch wenn er selbst nichts zeichnet. Fotografieren Sie beide Skizzen und committen Sie das Foto als `skizze-szenengraf.jpg`.

Bauen Sie die Oberfläche danach in `FreiraumApp`, verteilt auf zwei Branches:

- (A) Branch `a-listen`: beide Listen mit Überschriften in der Methode `private HBox listenBereich()`. Füllen Sie die Raumliste vorläufig mit `RaumDatei.lade(Path.of("daten/raeume.csv")).alle()`. Die `IOException` fangen Sie in `start` und zeigen einen Alert vom Typ `ERROR`.
- (B) Branch `b-eingabe`: Eingabezeile, Buttons und Statuszeile in der Methode `private VBox eingabeBereich()`. Jedes Textfeld bekommt mit `setPromptText` einen Platzhalter, etwa `"Beginn 10:00"`.

Mergen Sie beide Branches nach `main` und hängen Sie beide Bereiche in `start` unter eine gemeinsame Wurzel. Die Raumliste zeigt fünf Räume. Vergleichen Sie Ihren Baum mit der Skizze: Schreiben Sie die rekursive Methode `drucke(Node n, int tiefe)` aus T10 und rufen Sie `drucke(wurzel, 0)` vor `stage.show()` auf. Jede Abweichung zwischen Ausgabe und Papierskizze notieren Sie in `NOTIZEN.md`.

Dritte Aufgabe (50 Minuten): Legen Sie in `freiraum.logik` den Record `Buchungsergebnis` und die Klasse `Buchungssystem` an. Diese Köpfe sind vorgegeben, Testat 5 und Startstand 6 verwenden sie:

```
public record Buchungsergebnis(boolean erfolgreich, String meldung, boolean wartelisteMoeglich)
public static Buchungsergebnis ok(String meldung)           // wartelisteMoeglich false
public static Buchungsergebnis fehler(String meldung)       // wartelisteMoeglich false
public static Buchungsergebnis belegt(String meldung)       // wartelisteMoeglich true

public Buchungssystem(Campus campus)
public static Buchungssystem ausDateien(Path raeume, Path buchungen) throws IOException
public List<Raum> raeume()
public List<String> belegung(String raum)
public Buchungsergebnis buchen(String raum, String gruppe, String personen, String tag,
                                String beginn, String ende, boolean einweisung)
```

`buchen` nimmt die Texte aus den Eingabefeldern unverändert entgegen. Alles, was entscheidet, geschieht in dieser Methode: den Raum mit `Campus.finde` suchen, Personenzahl, Tag und Uhrzeiten parsen, Zeitfenster und Buchung erzeugen, `Raum.buchen` aufrufen. Keine Exception verlässt `buchen`. Fangen Sie `NumberFormatException`, `IllegalArgumentException` und `BuchungAbgelehntException` und liefern Sie `fehler(...)` mit einer Meldung, die mit Nicht gebucht: beginnt. Scheitert die Buchung nur an einer Überschneidung (`Raum.istFrei` liefert `false`), liefern Sie `belegt(...)`, denn die sechste Aufgabe braucht diese Unterscheidung. `ausDateien` lädt die Räume mit `RaumDatei.lade` und die Buchungen mit `BuchungsDatei.lade`; die `IOException` geht an den Aufrufer. `belegung` liefert die Buchungen des Raums als Textzeilen, früheste zuerst (`Raum.belegungChronologisch`), bei unbekanntem Raum eine leere Liste.

Ersetzen Sie in `start` das vorläufige Laden aus der zweiten Aufgabe durch `Buchungssystem.ausDateien(Path.of("daten/raeume.csv"), Path.of("daten/buchungen-woche.csv"))` und füllen Sie die Raumliste mit `raeume()`. Commit.

Vierte Aufgabe (50 Minuten): Registrieren Sie die Handler. Der Handler von „Buchen“ hat höchstens vier Anweisungen: Eingaben ablesen und an `system.buchen(...)` übergeben, die Meldung in der Statuszeile anzeigen, die Belegungsliste aktualisieren. Der Handler von „Belegung anzeigen“ setzt nur die Einträge der Belegungsliste neu (`getItems().setAll(...)`). Kein Handler parst, rechnet oder prüft: Wie der Kellner aus T10 trägt er die Bestellung nur in die Küche.

Das Buchungssystem halten Sie in einem Attribut von `FreiraumApp`. Wer im Handler eine lokale Variable von außerhalb verändert, bekommt vom Compiler error: `local variables referenced from a lambda expression must be final or effectively final`.

Bedienen Sie die Oberfläche mit den folgenden fünf Eingaben und kopieren Sie jede Statusmeldung in `NOTIZEN.md`:

Raum	Gruppe	Personen	Tag	Beginn	Ende	Einweisung
LR-1	Team Blau	4	1	10:00	11:00	nein
LR-1	Team Rot	vier	1	12:00	13:00	nein
LR-1	Team Rot	3	1	09:00	10:00	nein
UX-LAB	Team Grün	2	4	08:00	09:00	nein
keiner gewählt	Team Grün	2	4	08:00	09:00	nein

Nach der ersten Zeile zeigt die Statuszeile auf dem Referenzstand Gebucht in LR-1: #41 Mo 10:00–11:00 Team Blau, 4 Pers., die Belegung von LR-1 hat 11 Zeilen und beginnt mit #23 Mo 08:30–09:30 Team Blau, 11 Pers.

Prüfen Sie jede ablehnende Meldung gegen die drei Fragen aus T06B: Was ist passiert? Was heißt das für mich? Was kann ich jetzt tun? Fehlt eine Antwort, ändern Sie den Text in `Buchungssystem`, nicht im Handler. So kann die Meldung zur zweiten Zeile aussehen: Nicht gebucht: „vier“ ist keine Personenzahl – bitte Ziffern eingeben, zum Beispiel 4.

Schreiben Sie den Handler von „Buchen“ Zeile für Zeile in `NOTIZEN.md` und ordnen Sie jede Zeile einer Schicht zu: Oberfläche, Handler oder Logik. Commit.

Fünfte Aufgabe (30 Minuten): Schreiben Sie in `freiraum.logik` die Klasse `PruefungLogik`. Ihr `main` lädt ein Buchungssystem aus den beiden Dateien und prüft mit `pruefe` die fünf Zeilen der Tabelle aus der vierten Aufgabe: für jede Zeile `erfolgreich()` und `wartelisteMoeglich()`, für die letzte Zeile zusätzlich den vollständigen Meldungstext. Kein `launch`, keine `Stage`. Starten Sie das Programm ohne JavaFX auf dem Klassenpfad:

```
mvn compile
java -cp target/classes freiraum.logik.PruefungLogik
```

Machen Sie danach die Gegenprobe: Zeigen Sie testweise in `Buchungssystem.buchen` einen Alert, kompilieren Sie und starten Sie `PruefungLogik` mit demselben Befehl. Tragen Sie die erste Zeile der Fehlermeldung wörtlich in `NOTIZEN.md` ein. Auf dem Referenzrechner lautet sie `Exception in thread "main" java.lang.NoClassDefFoundError: javafx/scene/control/Alert`. Startet Ihre IDE das Programm mit JavaFX, erscheint eine andere Meldung; notieren Sie die, die Sie sehen. Nehmen Sie die Änderung mit `git restore` zurück. `PruefungLogik` endet danach wieder mit 0 FEHLER.

Sechste Aufgabe (40 Minuten): Ergänzen Sie `Buchungssystem` um diese Methode:

```
public Buchungsergebnis aufWarteliste(String raum, String gruppe, String personen, String tag,  
                                     String beginn, String ende, boolean einweisung)
```

Sie erzeugt die Buchung wie buchen, stellt sie mit `Raum.aufWarteliste(Buchung)` an und nennt die zurückgegebene Position in der Meldung. Liefert buchen ein Ergebnis mit `wartelisteMoeglich()`, fragt die Oberfläche mit einem Alert vom Typ `CONFIRMATION` und den Knöpfen `ButtonType.YES` und `ButtonType.NO`, ob die Buchung auf die Warteliste soll. `showAndWait()` liefert die Antwort als `Optional<ButtonType>`. Bei „Ja“ ruft die Oberfläche `aufWarteliste` auf und zeigt deren Meldung in der Statuszeile. Den Dialog baut eine eigene Methode in `FreiraumApp`; der Handler von „Buchen“ bleibt bei vier Anweisungen.

Prüfen Sie mit der dritten Tabellenzeile: Team Rot steht auf Position 1 der Warteliste von LR-1. Nehmen Sie diesen Fall in `PruefungLogik` auf. Commit.

Siebte Aufgabe (30 Minuten): Sabotage-Probe im Paar, nach dem Muster aus T10A.

- (A) legt den Branch `sabotage-a` an und baut, ohne es B zu zeigen, genau eine Sabotage in `FreiraumApp` ein: einen Handler am falschen Knoten registrieren, einen Handler entfernen oder das Aktualisieren einer Liste weglassen. Commit, `git push -u origin sabotage-a`.
- (B) holt den Branch mit `git fetch` und `git switch sabotage-a`, startet die Oberfläche und sucht die Sabotage nur durch Bedienen. B notiert zuerst das Symptom aus Benutzersicht in einem Satz, dann die vermutete Codestelle, und prüft die Vermutung erst danach mit `git diff main sabotage-a`.

Tauschen Sie die Rollen (Branch `sabotage-b`). Notieren Sie in `NOTIZEN.md` je Sabotage, ob Java eine Meldung ausgegeben hat. Die Sabotage-Branches werden nicht gemergt: Löschen Sie sie mit `git branch -D` und `git push origin --delete`. Setzen Sie auf `main` den Tag `abgabe-5` und pushen Sie ihn mit `git push origin abgabe-5`.